

I N D E X

Name Altamash Akhwar

Std 4th Sem Sec

Roll No. Subject

School/College

SL No.	Date	Title	Page No.	Teacher Sign / Remarks
1	2/3	FCFS, SJF		<i>[Signature]</i> 2/3/26
2	10/3	Priority		<i>[Signature]</i> 10/3/26
3	10/3	Round Robin		<i>[Signature]</i> 10/3/26
4	24/3	Multilevel Queue Scheduling		<i>[Signature]</i> 24/3/26
5	31/3	EDF, RMS		
6		a) Producer - Consumer		
		b) Dining Philosophers		
7		Banker's algorithm		
8		Memory Allocation techniques		
9		Page Replacement:		
		(a) FIFO		
		(b) LRU		
		(c) Optimal		

Program 1: Find the second smallest & second largest element in an Array

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i;
```

```
    printf ("Enter number of elements: ");
```

```
    scanf ("%d", &n);
```

```
    int arr[n];
```

```
    printf ("Enter elements: \n");
```

```
    for (i=0; i<n; i++) scanf ("%d", &arr[i]);
```

```
    int smallest = arr[0], second = arr[0],  
        large = arr[0];
```

```
    for (i=1; i<n; i++) {
```

```
        if (arr[i] < smallest) {
```

```
            second = smallest;  
            smallest = arr[i];
```

```
        }  
        if (arr[i] > largest) {
```

```
            large = largest;  
            largest = arr[i];
```

```
        }
```

```
        else if (arr[i] < second && arr[i] != smallest)
```

```
            second = arr[i];
```

```
    }
```

```
    else if (arr[i] < second && arr[i] != smallest)
```

```
        second = arr[i];
```

```
    else if (arr[i] > largest && arr[i] != large)
```

```
        large = arr[i];
```

```
}
```

printf("%d", second_smallest_element);
return 0;

3
O/P

Enter elements: 1 2 3 4 5 6

second smallest element: 2

second largest element: 5

~~3/19/26~~

Program 2:

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++)
    {
        printf("Enter element %d: ", i+1);
        scanf("%d", &arr[i]);
    }
}
```

O/P

Enter n:

Enter element

Sum of

Second

Program 2: Find sum of left diagonal of a matrix

```
#include <stdio.h>
int main () {
    int n, i, j, sum=0;
    printf("Enter order of square matrix: ");
    scanf("%d", &n);
    int matrix[n][n];
    printf("Enter matrix elements: \n");
    for (i=0; i<n; i++) {
        for (j=0; j<n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }
    for (i=0; i<n; i++) {
        sum += matrix[i][i];
    }
    printf("Sum of left diagonal = %d", sum);
    return 0;
}
```

O/P

Enter order of square matrix: 2

Enter matrix elements: 1 2 3 4

Sum of left diagonal elements: 5

```
# include <stdio.h>
int main ()
```

```
int r, c, i, j;
```

```
int matrix [2][2] = { {1, 2}, {3, 4} },
```

```
r = 2, c = 2;
```

```
for (i = 0; i < r; i++)
```

```
int rsum = 0;
```

```
for (j = 0; j < c; j++)
```

```
rsum = mat[i][j];
```

```
printf ("Sum of row %d = %d\n", i+1, rsum);
```

```
}
```

```
for (j = 0; j < c; j++)
```

```
int csum = 0;
```

```
for (i = 0; i < r; i++) csum += mat[i][j];
```

```
printf ("Sum of column %d = %d\n", j+1, csum);
```

```
}
```

```
return 0;
```

```
}
```

O/P

Sum of Row 1 : 3

Sum of Row 2 : 7

Sum of column 1 : 4

Sum of column 2 : 6

Program 4: count total numbers of duplicate elements in an array

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, j, count = 0;
```

```
    int arr[5] = {1, 1, 3, 4, 5};
```

```
    for (i = 0; i < n; i++) {
```

```
        for (j = i + 1; j < n; j++) {
```

```
            if (arr[i] == arr[j]) {
```

```
                count++;
```

```
                break;
```

```
            }
```

```
        }
```

```
    printf ("Total duplicates : %d", count);
```

```
    return 0;
```

```
}
```

O/P

Total duplicates: 2

Program 5: Delete an element at desired position

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, pos;
```

```
    int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
    n = 10;
```

```
    printf ("Enter the position to delete: ");
```

```
    scanf ("%d", &pos);
```

```
    if (pos < 1 || pos > n) {
```

```
        printf ("Invalid position");  
        return 0;
```

```
    }
```

```
    for (i = pos - 1; i < n - 1; i++) arr[i] = arr[i + 1];
```

```
    n--;
```

```
    printf ("Array after deletion: \n");
```

```
    for (i = 0; i < n; i++) printf ("%d", arr[i]);
```

```
    return 0;
```

```
}
```

Enter the position to delete: 5

Array after deletion:

1 2 3 4 6 7 8 9 10

Programs Implementing FCFS Scheduling (both preemptive and non-preemptive) and compare which scheduling process is more effective.

Code:

```
#include <stdio.h>
#include <limits.h>
struct Process {
    int pid, at, bt, ct, tat, wt,
    remaining_bt;
```

```
};
```

```
float fcfs (struct Process p[], int n) {
```

```
    int time = 0;
```

```
    float total_wt = 0, total_tat = 0;
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (p[i].at > p[j].at) {
```

```
                struct Process temp = p[i];
```

```
                p[i] = p[j];
```

```
                p[j] = temp;
```

```
            }
        }
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (time < p[i].at) time = p[i].at;
```

```
        p[i].ct = time + p[i].bt;
```

```
        p[i].wt = p[i].ct - p[i].at;
```

```
        p[i].tat = p[i].ct - p[i].bt;
```



```

total_wt += p[i].wt;
total_tat += p[i].tat;
time = p[i].ct;

```

```

}
printf (" In FCFS In ");
printf (" Process | TAT | BT | CT | TAT |  

        | WT | ");
for (int i=0; i<n; i++)
    printf ("%d | %d | %d | %d | %d |  

            %d | ",
            p[i].pid, p[i].at, p[i].bt,
            p[i].ct, p[i].tat, p[i].wt);

printf (" Average WT = %.2f In ", total_wt/n);
printf (" Average TAT = %.2f In ", total_tat/n);
return total_wt/n;
}

```

Next sig - non-preemptive (struct process p[], int n);

```

int completed = 0, time = 0;

```

```

int visited [n];

```

```

float total_wt = 0, total_tat = 0;

```

```

for (int i=0; i<n; i++) if (visited[i] == 0,

```

```

while (completed < n) {

```

```

    int minBT = INT_MAX, index = -1;

```



```

for (int i = 0 ; i < n ; i++)
    if (p[i].at <= time && ! visited[i])
        if (p[i].bt < minBT)
            minBT = p[i].bt;
            index = i;

```

```

if (index != -1)
    time += p[index].bt;
    p[index].ct = time;
    p[index].tat = p[index].ct - p[index].at;
    p[index].wt = p[index].tat - p[index].bt;
    total_wt += p[index].wt;
    total_tat += p[index].tat;
    visited[index] = 1;
    completed++;

```

```

} else time++;

```

```

}
printf("\n SFC. (Non-Preemptive) \n");
printf("\n Process | TAT | BT | CT | TAT | WT \n");

```

```

for (int i = 0 ; i < n ; i++)

```

```

    printf("%d | %d | %d | %d | %d | %d \n",
           p[i].pid, p[i].at, p[i].bt, p[i].ct,
           p[i].tat, p[i].wt);

```



```

printf "Average WT = %.2f\n", total-wt/n;
printf "Average TAT = %.2f\n", total-tat/n;
return total-wt/n;

```

3

```

float sjf - preemptive (struct process p[n],
                        int n)

```

```

int completed = 0, time = 0;
float total-wt = 0, total-tat = 0;

```

```

for (int i = 0; i < n; i++)
    p[i].remaining-bt = p[i].bt;

```

```

while (completed < n)

```

```

    int minBT = INT_MAX, index = -1;

```

```

    for (int i = 0; i < n; i++)

```

```

        if (p[i].at <= time && p[i].

```

```

            remaining-bt > 0)

```

```

            minBT = p[i].remaining-bt;

```

```

            index = i;

```

```

    if (index != -1)

```

```

        p[index].remaining-bt--;
        time++;

```


total - w/ (y)
total - tot (n)

100 p57,
+n) 2

$$t = 0,$$

psid. b7;

x , index = -1;

$i + + f \checkmark$

Am 21. Juli.

3

Remains - 6/20

remaining - 1st;

--- 44 ---

3

print ("Average WT = %.2f \n", total - tot / n);

using 1/16" diameter TAT = $\gamma \cdot R / \ln''$, total - tat / $\ln''$

return total - wt / n;


```
int main () {
```

```
    int n;  
    printf ("Enter number of processes ");
```

```
    scanf ("%d", &n);
```

```
    struct Process original [n];
```

```
    for (int i=0; i<n; i++) {
```

```
        original [i].pid = i+1;
```

```
        printf ("Enter Arrival Time and  
        Burst Time for P%d", i+1);
```

```
        scanf ("%d %d", &original [i].at,  
        &original [i].bt);
```

```
    }  
    struct Process p1 [n], p2 [n], p3 [n];
```

```
    for (int i=0; i<n; i++) {
```

```
        p1 [i] = original [i];
```

```
        p2 [i] = original [i];
```

```
        p3 [i] = original [i];
```

```
    }  
}
```

```
float fcfs-wt = fcfs (p1, n);
```

```
float sjf-np-wt = sjf-non-preemptive  
                (p2, n);
```

```
float sjf-p-wt = sjf-preemptive (p3, n);
```



```
printf ("Inefficiently order (best to worst) \n");  
if (sif - p-wt <= sif - np-wt || sif - npwt  
    <= /cp-wt)
```

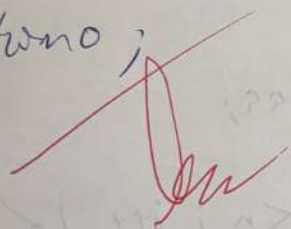
```
printf ("SDF (Presumptive) >  
SDF (Non presumptive)  
> FCFS \n");
```

```
else if (sif - np-wt <= sif - p-wt ||  
sif - p-wt <= /cp-wt)
```

```
printf ("SDF (Non-presumptive) >  
SDF (presumptive) >  
FCFS \n");
```

```
else {  
printf ("FCFS is least efficient  
among the three. \n");
```

```
return 0;
```



Program 2:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
→ priority (pre-emptive and Non pre-emptive).

```
#include <stdio.h>
```

```
struct process {
```

```
    int pid, arrived, burst, priority;
```

```
    int remaining, completion, turnaround, waiting;
```

```
    int finished;
```

```
};
```

```
void nonPreemptive (struct process p[], int n) {
```

```
    int time = 0, completed = 0;
```

```
    float totalTAT = 0, totalWT = 0;
```

```
    while (completed < n) {
```

```
        int idx = -1;
```

```
        int highest = 9999;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (p[i].arrived <= time & p[i].finished == 0)
```

```
                if (p[i].priority < highest) {
```

```
                    highest = p[i].priority;
```

```
                    idx = i;
```


$$1/ (id_X \circ -) = -1 \circ$$

time $t = \rho \cdot \text{idx}$. lower;

$p \text{ rad} \times J. \text{ completion} = \text{Time};$

$\text{turnaround} = p[id.x].\text{completion} - p[id.x].\text{arrival};$

$$p(\text{fid} \times J, \text{wetting}) = p(\text{fid} \times J, \text{turnaround}) - p(\text{fid} \times J, \text{burst})$$

total TAT = $p[id \times]$ turns around;
5.2 waiting;

total WT $t = [p \times \text{tid} \times 2]$ waiting;

$p \text{ } \tau(dx).finished = 1;$

completed ++;

or else &

time + + i'

3

```
printf("In PID |+AT |+BT |+PR |+CT |  
      |+TAT |+WT |n");
```

```
for (int i = 0; i < n; i++) {
```

```
for (int i = 0; i < n; i++)
    printf("%d\t", arr[i]);
```

prj. prod, prj. arrival, prj. burst,
prj. completion,

priority, completion, waiting);

pr_i: turning, pr_i: waiting);

3


```
printf ("In Average Turnaround Time = %.2f",  
totalTAT / n);
```

```
printf ("In Average Waiting Time = %.2f",  
totalWT / n);
```

```
void preemptive (struct process p[], int p)
```

```
{  
    int time = 0, completed = 0;
```

```
    float totalTAT = 0, totalWT = 0;
```

```
    while (completed < n) {
```

```
        int idx = -1;
```

```
        int highest = 9999;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (p[i].arrival <= time & p[i].remaining > 0) {
```

```
                if (p[i].priority < highest) {
```

```
                    highest = p[i].priority;
```

```
                    idx = i;
```

```
            }
```

```
        }
```

```
        if (idx != -1) {
```

```
            p[idx].remaining--;
```

```
            time++;
```

```
            if (p[idx].remaining == 0) {
```

```
                completed++;
```


2 p" -
12"

$p) \alpha$

14

```

int main () {
    int n, choice;
    printf("Enter number of processes : ");
    scanf("%d", &n);
    struct process p[10];
    for (int i = 0; i < n; i++) {
        printf("Process %d : ", i+1);
        p[i].pid = i+1;
        printf("Arrival Time: ");
        scanf("%d", &p[i].arrival);
        printf("Burst Time: ");
        scanf("%d", &p[i].burst);
        printf("Priority : ");
        scanf("%d", &p[i].priority);
        p[i].remaining = p[i].burst;
        p[i].finished = 0;
    }
}

```

```

// 10 Process Scheduling Algorithm in C
// 1. Priority Non-preemptive in C
// 2. Round Robin Preemptive in C
// 3. Round Robin Preemptive in C
// 4. Round Robin Preemptive in C
// 5. Round Robin Preemptive in C
// 6. Round Robin Preemptive in C
// 7. Round Robin Preemptive in C
// 8. Round Robin Preemptive in C
// 9. Round Robin Preemptive in C
// 10. Round Robin Preemptive in C

```

```

if (choice == 1) {
    nonPreemptive
} else if (choice == 2) {
    preemptive
} else {
    printf("Invalid choice\n");
    return 0;
}

```

o/p

Enter number of processes : 5
 Process 1
 Arrival Time : 25
 Burst Time : 45
 Priority : 1

Process 2
 Arrival Time : 30
 Burst Time : 25
 Priority : 2

Process 3
 Arrival Time : 35
 Burst Time : 30
 Priority : 3


```

if (choice == 1)
    nonPreemptive(p, n);
else if (choice == 2)
    preemptive(p, n);
else
    printf("Invalid choice\n");
return 0;

```

O/P

Enter number of processes;
3

Process 1

Arrival Time : 23

Burst Time : 45

Priority : 1

Process 2

Arrival Time : 67

Burst Time : 89

Priority : 3

Process 3

Arrival Time : 78

Burst Time : 39

Priority : 3

10/3/26

Choose Scheduling Algorithm

1. Priority Non-Preemptive

2. Priority Preemptive

Enter choice : 1

PID	AT	BT	PR	CT	TAT	WT
1	23	45	1	68	45	0
2	67	89	3	257	90	1
3	78	39	3	126	118	79

Average Turnaround Time = 84.33

Average Waiting Time = 26.67

Output for Program 1

For FCFS

Enter number of processes : 3

Enter AT and BT for P1: 0 5

Enter AT and BT for P2: 3 2

Enter AT and BT for P3: 2 1

Process	AT	BT	CT	WT	TAT	RT
P1	0	5	5	0	5	0
P3	2	1	6	3	4	3
P2	3	2	8	3	5	3

Average WT = 2.00

Average TAT = 4.67

Average CT = 6.33

Average RT = 2.00

For SJF

Enter number of processes

Enter AT and BT

Enter AT and BT

Enter AT and BT

1. SJF Non preemptive

2. SJF Preemptive (S)

Enter choice : 1

PID AT BT

P1 0 5

P2 3 2

P3 2 1

Average WT = 2.00

Average TAT = 4.67

Average RT = 2.00

Average WT = 2.00

Average TAT = 4.67

Average CT = 6.33

Average RT = 2.00

For SJF =

Enter number of processes: 3

Enter AT and BT for P1: 0 5

Enter AT and BT for P2: 3 2

Enter AT and BT for P3: 2 1

1. SJF Non preemptive

2. SJF Preemptive (SRTF)

Enter choice: 1

PID	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	3	2	8	5	3	3
P3	2	1	6	4	3	3

Average WT = 2.00

Average TAT = 4.67

Average RT = 2.00

17/03/26

Program 3: Round Robin

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, time = 0, remain, quantum;
```

```
    int wait_time = 0, turnaround_time = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int bt[n], rt[n];
```

```
    printf("Enter burst time for each process: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("p%d: ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
```

```
    }
```

```
    printf("Enter time quantum: ");
```

```
    scanf("%d", &quantum);
```

```
    remain = n;
```

```
    int t = 0;
```

```
    printf("\n Process \t Turnaround Time \t Waiting Time \n");
```

```
    for (i = 0; remain != 0; ) {
```

```
        if (rt[i] <= quantum && rt[i] > 0) {
```

```
            t += rt[i];
```

```
            rt[i] = 0;
```

```
            remain--;
```



```

int tat = 0;
int wt = tat - t[i]
turnaround_time = t = tat;
wait_time += wt;
printf("p%d | t%d | t+d | t+d\n",
       i+1, tat, wt);
}
else if (t[i] > 0) {
    t[i] -= quantum;
    t += quantum;
}
if (i == n - 1)
    i = 0;
else
    i++;
}
printf("\n Average turnaround Time = %.2f",
       (float) turnaround_time / n);
printf("\n Average Waiting Time = %.2f\n",
       (float) wait_time / n);
return 0;
}

```


O/P:

Enter number of processes : 2

Enter burst time for each process:

P1: 30

P2: 67

Enter time quantum : 3

Process	Turnaround Time	Waiting Time
P1	57	27
P2	97	30

Average Turnaround Time = ~~72.00~~

Average Waiting Time = 28.50

Program - 4

~~write C program~~
scheduling algorithm

~~az Rate Monotonic~~

~~az Multilevel~~

#include <stdio.h>

#define MAX 20

typedef struct t
{
int pid;
int burst;
int remaining;
int waiting;
int turnaround;

Process;

void fcf (P

int n;

for (int i

if

tim

pr

pr

pr

Process:

Program - 4
C program to simulate Real Time CPU
with scheduling algorithms.

1. ~~2. Rate Monotonic~~

2. ~~3. Multilevel Queue Scheduling~~

#include <stdio.h>

#define MAX 20

typedef struct t
{
int pid;
int arrival;
int burst;
int completion;
int waiting;
int turnaround;

} Process;

void fcts (Process p[], int n, int start_time) {
int time = start_time;

for (int i = 0; i < n; i++) {

if (time < p[i].arrival) {
time = p[i].arrival;

time += p[i].burst;

p[i].completion = time;

p[i].turnaround = p[i].completion - p[i].arrival;

p[i].waiting = p[i].turnaround - p[i].burst;

}

```

void display (Process p[], int n, char queue-name) {
    printf ("In %s Queue : \n", queue-name);
    printf ("PID | AT | BT | CT | TAT | WT | \n");
    for (int i = 0; i < n; i++) {
        printf ("%d | %d | %d | %d | %d | %d | \n",

```

```

            p[i].pid,
            p[i].arrived,
            p[i].burst,
            p[i].completion,
            p[i].turnaround,
            p[i].waiting);
    }
}

```

```

int main () {

```

```

    int n, sys-count = 0, user-count = 0;

```

```

    Process system [MAX], user [MAX];

```

```

    printf ("Enter total number of processes: ");

```

```

    scanf ("%d", &n);

```

```

    for (int i = 0; i < n; i++) {
        int type;

```

```

        Process temp;

```

```

        printf ("In Process %d \n", i+1);
    }
}

```

```

printf ("
scanf ("
printf
scanf ("
printf
scanf ("

```

```

temp.pro
if (type
} else

```

```

}
}
fco (system,

```

```

int last-time
if (sys-count
last-ti

```

```

fco / user,

```

```

display (system
display (user

```

```

return 0;
}

```


sehen 0;

 $i+1$);

O/P total number of processes: 3

Process 1

Enter Arrival Time: 45

Enter Burst Time: 67

Enter Type (1-System, 2-User): 1

Process 2

Enter Arrival Time: 23

Enter Burst Time: 78

Enter Type (1-System, 2-User): 2

Process 3

Enter Arrival Time: 23

Enter Burst Time: 65

Enter Type (1-System, 2-User): 2

System Queue:

PID	AT	BT	CT	TAT	WT
1	45	67	112	67	0

User Queue:

PID	AT	BT	CT	TAT	WT
2	23	78	120	167	89
3	23	65	255	232	167

Program 5

```
#include <stdio.h>
```

```
#define MAX 10
```

```
typedef struct {
    int id,
    int ct,
    int done;
```

```
} Process;
```

```
Process p[MAX];
```

```
int n;
```

```
void edf(.) {
    Process p;
```

```
for (int i = 0; i < n; i++)
```

```
int time;
```

```
while (1)
```

```
int
```

```
int
```

```
for
```


Program 5

```
#include <stdio.h>
```

```
#define MAX 10
```

```
typedef struct t
```

```
{ int id, bt, deadline, period;
```

```
  int ct, wt, tat;
```

```
  int done;
```

```
} Process;
```

```
Process p[MAX];
```

```
int n;
```

```
void edf() {
```

```
  Process temp[MAX];
```

```
  for (int i=0; i<n; i++) {
```

```
    temp[i] = p[i];
```

```
    temp[i].done = 0;
```

```
  }  
  int time = 0, completed = 0;
```

```
  while (completed < n) {
```

```
    int idx = -1;
```

```
    int min_deadline = 9999;
```

```
    for (int i=0; i<n; i++) {
```

```
      if (!temp[i].done && temp[i].deadline < min_deadline)
```

```
        min_deadline = temp[i].deadline;
```

```
        idx = i;
```

```
    }
```

```

time += temp[i].bt;
temp[i].ct = time;
temp[i].tot = temp[i].ct;
temp[i].wt = temp[i].tot - temp[i].bt;
temp[i].done = 1;
completed++;

```

```

}
for (int i=0; i<n-1; i++) {
    for (int j=i+1; j<n; j++) {
        if (temp[i].deadline > temp[j].deadline) {
            process = temp[i];
            temp[i] = temp[j];
            temp[j] = process;
        }
    }
}

```

```

}
printf("\n==== Earliest Deadline First (EDF) Scheduling ====\n");
printf("ID | BT | Deadline | CT | WT | TAT | \n");
for (int i=0; i<n; i++) {
    printf("%d | %d | %d | %d | %d | %d | \n",
        temp[i].id, temp[i].bt, temp[i].deadline,
        temp[i].ct, temp[i].wt, temp[i].tot);
}

```


void rms () {

Process temp [MAX];

for (int i=0; i<n; i++) {

temp[i] = p[i];

temp[i].done = 0;

}

int time = 0, completed = 0;

while (completed < n) {

int idx = -1;

int min-period = 9999;

for (int i=0; i<n; i++) {

if (!temp[i].done && temp[i].period < min-period) {

min-period = temp[i].period;

idx = i;

}

time += temp[idx].wt;

temp[idx].ct = time;

temp[idx].tot = temp[idx].ct;

temp[idx].wt = temp[idx].tot - temp[idx].bt;

temp[idx].done = 1;

completed ++;

}


```

for (int i=0; i<n-1; i++) {
    for (int j=i+1; j<n; j++) {
        if (temp[i].period > temp[j].period) {
            Process t = temp[i];
            temp[i] = temp[j];
            temp[j] = t;
        }
    }
}

```

```

printf("\n ===== Rate Monotonic Scheduling\n\n\n");

```

```

printf("\n ID \t BT \t Period \t CT \t WT \t TBT\n");

```

```

for (int i=0; i<n; i++) {

```

```

    printf("\nEnter number of processes: ");

```

```

    scanf

```

```

    printf("\n %d \t %d \t %d \t %d \t %d \n",
        temp[i].id, temp[i].bt, temp[i].period,
        temp[i].ct, temp[i].wt, temp[i].tbt);
}

```

```

int main () {

```

```

    printf("\nEnter number of processes: ");

```

```

    scanf("%d", &n);

```

```

    printf("\nEnter process details: \n");

```

O/P

Enter number

Enter process

Process 0:

Burst Time:

Deadline (for

Period (for R

17% of 1 in",
 1st, temp. 12. 1st, 1st,
 1st, temp. 12, 1st,

h'')

```

for (int i=0; i<n; i++) {
    p[i].id = i;
    printf("In Process %d : %d", i);
    printf("Burst Time : ");
    scanf("%d", &p[i].bt);

    printf("Deadline (for EDF): ");
    scanf("%d", &p[i].deadline);

    printf("Period (for RMS): ");
    scanf("%d", &p[i].period);
}

```

3
colf ();
fms ();
setzen 0;

7

0/p

Enter number of processes : 3

Enter process details:

Process 0:

Burst Time : 34

Deadline (for EDF): 89

Period (for RMS): 56

Process 1:
 Burst Time: 24
 Deadline (for EDF): 99
 Period (for RMS): 63

Process 2:
 Burst Time: 12
 Deadline (for EDF): 18
 Period (for RMS): 29

==== Earliest Deadline First (EDF) Scheduling ====

ID	BT	Deadline	CT	WT	TAT
2	12	18	12	0	22
0	34	89	46	12	46
1	24	99	70	46	70

==== Rate Monotonic Scheduling (RMS) ====

ID	BT	Period	CT	WT	TAT
2	12	29	12	0	12
0	34	56	46	12	46
1	24	63	70	46	70

Done 31/3/26

EDF

P_2



RMS

Consider two processes A and B. Two semaphore variable S and T are used to synchronize the processes. S is initialized to 0 and T is initialized to 1. Processes are scheduled in the following order: AB ABA ABA.

What will be printed on the screen?

Process A:
while(1)
{

wait(S);
print 'P';
print 'P';
signal(T);
}

Process B:
while(1) {

wait(T);
print(1);
print(1);
signal(S);
}

	S	T	Remark
initial	S=0	T=1	
A	0	1	Process A is blocked
B	1	0	11
A	0	1	12PP
B	1	0	11PP11
B	1	0	Process B is blocked

A	0
A	0
B	1
A	0

Program 6

Write a C program to simulate;

- a) Producer - Consumer problem using semaphores
- b) Dining - Philosopher's problem.

Output

Produced: 42 at position 0

Consumed: 42 from position 0

Produced: 67 at position 1

Consumed: 67 from position 1

Produced: 34 at position 2

Produced: 0 at position 3

Consumed: 34 from position 2

Produced: 69 at position 4

Produced: 24 at position 0

Consumed: 0 from position 3

Produced: 78 at position 1

Produced: 58 at position 2

Consumed: 69 from position 4.

Tracing

Initial:

Buffer = [-, -, -, -, -]

in = 0

out = 0

by semaphore

① [41, -, -, -, -] in = 1 out = 0

② [-, -, -, -, -] in = 1 out = 2

③ [-, 67, -, -, -]

[Signature]
2/14/26

a) Producers - Consumers

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
#define BUFFER_SIZE 5
```

```
int buffer [BUFFER_SIZE];
int in=0, out=0;
```

```
sem_t empty, full, mutex;
```

```
void *producer (void *arg) {
```

```
    int item;
```

```
    while (1) {
```

```
        item = rand() % 100;
```

```
        sem_wait (&empty);
```

```
        sem_wait (&mutex);
```

```
        buffer [in] = item;
```

```
        printf ("Produced : %.d at position %.d\n",
                item, in);
```

```
        in = (in + 1) % BUFFER_SIZE;
```

```
        sem_post (&mutex);
```

```
        sem_post (&full);
```

```
        sleep(1);
```

```
    }
```

```
void *consumer (void *arg) {
    int item;
    while (1) {
```

```
        sem_wait (&full);
```

```
        sem_wait (&mutex);
```

```
        item = buffer [out];
```

```
        printf ("Consumed : %.d\n", item);
```

```
        out = (out + 1) % BUFFER_SIZE;
```

```
        sem_post (&mutex);
```

```
        sem_post (&empty);
```

```
        sleep(1);
```

```
    }
```

```
}

int main () {
    pthread_t p, c;
```

```
    sem_init (&empty, 0, 5);
```

```
    sem_init (&full, 0, 0);
```

```
    sem_init (&mutex, 0, 1);
```

```
    pthread_create (&p, NULL, producer, NULL);
```

```
    pthread_create (&c, NULL, consumer, NULL);
```

```
    pthread_join (p, NULL);
```

```
    pthread_join (c, NULL);
```

```
    return 0;
```

```
}
```



```
void * consumer (void *arg) {
```

```
    int item;
```

```
    while (1) {
```

```
        sem_wait (&full);
```

```
        sem_wait (&mux);
```

```
        item = buffer[out];
```

```
        printf("Consumed: %d from position %d\n",  
              item, out);
```

```
        out = (out + 1) % BUFFER_SIZE;
```

```
        sem_post (&mux);
```

```
        sem_post (&empty);
```

```
        sleep(2);
```

```
    }
```

```
}
```

```
int main () {
```

```
    pthread_t p, c;
```

```
    sem_init (&empty, 0, BUFFER_SIZE);
```

```
    sem_init (&full, 0, 0);
```

```
    sem_init (&mux, 0, 1);
```

```
    pthread_create (&p, NULL, producer, NULL);
```

```
    pthread_create (&c, NULL, consumer, NULL);
```

```
    pthread_join (p, NULL);
```

```
    pthread_join (c, NULL);
```

```
    return 0;
```

```
}
```


b) Dining - Philosophers

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t forks[N];

void *philosopher (void *num) {
    int id = *(int *) num;
    while (1) {
        printf("Philosopher %d is thinking. \n", id);
        sleep(1);

        sem_wait(&forks[(id+1)%N]);
        printf("Philosopher %d is picked up  
right fork %d. \n", id, (id+1)%N);

        sem_wait(&forks[id]);
        printf("Philosopher %d is picked up left  
fork %d. \n", id, id);

        printf("Philosopher %d is eating. \n", id);
        sleep(2);

        sem_post(&forks[id]);
        sem_post(&forks[(id+1)%N]);
        printf("Philosopher %d put down forks  
%d and %d. \n", id, id, (id+1)%N);
    }
}
```

```
int main () {
    pthread_t phil;
    for (int i = 0; i < N; i++) {
        pthread_create(&phil, NULL, philosopher, (void *) i);
    }
    return 0;
}
```

Off Philosopher
Philosopher
philosopher
philosopher
Philosopher
Philosopher
Philosopher


```

int main () {
    pthread_t thread_id[N];
    int phil [N];

    for (int i=0; i<N; i++) {
        sem_init (&forks[i], 0, 1);
    }

    for (int i=0; i<N; i++) {
        phil[i] = i;
        pthread_create (&thread_id[i], NULL,
            philosopher, &phil[i]);
    }

    for (int i=0; i<N; i++) {
        pthread_join (thread_id[i], NULL);
    }

    return 0;
}

```

21/4/26

Philosopher 1 is thinking
 Philosopher 1 picked up right fork 2
 Philosopher 0 picked up right fork 1
 Philosopher 4 picked up right fork 0
 Philosopher 4 picked up right fork 4.
 Philosopher 3 picked up left fork 3.
 Philosopher 3 picked up left fork 3.
 Philosopher 3 is eating

forks
 (id+1)%N);

Program 7

Q) Banker's algorithm

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int Allocation[n][m], Max[n][m], Need[n][m],
```

```
    Available[m];
```

```
    int Finish[n], SafeSequence[n],
```

```
    int Work[m];
```

```
    printf("Enter Allocation Matrix (%d x %d): \n", n, m);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &Allocation[i][j]);
```

```
    printf("Enter Max Matrix (%d x %d): \n", n, m);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &Max[i][j]);
```

```
    printf("Enter Available Resources (%d): \n", m);
```

```
    for (int j = 0; j < m; j++)
```

```
        scanf("%d", &Available[j]);
```

```
    for (int j = 0;
```

```
        Work[j];
```

```
    for (int i =
```

```
        Finish[i];
```

```
    for (int i =
```

```
        for (i
```

```
        N
```

```
    int count =
```

```
    while (count
```

```
        int four
```

```
        for (in
```

```
        if
```



```
for (int j=0 ; j<m ; j++)  
    Work [j] = Available [j];
```

```
for (int i=0 ; i<n ; i++)  
    Finish [i] = 0;
```

```
for (int i=0 ; i<n ; i++)  
    for (int j=0 ; j<m ; j++)  
        Need [i] [j] = Max [i] [j] - Allocation [i] [j];
```

```
int count = 0;
```

```
while (count < n) {
```

```
    int found = 0;
```

```
    for (int i=0 ; i<n ; i++) {
```

```
        if (Finish [i] == 2) {
```

```
            int canRun = 1;
```

```
            for (int j=0 ; j<m ; j++) {
```

```
                if (Need [i] [j] > Work [j]) {
```

```
                    canRun = 0;
```

```
                    break;
```

```
            }  
            if (canRun) {
```

```
                for (int j=0 ; j<m ; j++)
```

```
                    Work [j] += Allocation [i] [j];
```

```
                Finish [i] = 1;
```

```
                Safe Sequence [count++] = i;
```

```
                found = 1;
```

```

if (!found) {
    printf("System is in an UNSAFE STATE (Potential Deadlock). \n");
    return 0;
}

```

```

printf("System is in a SAFE STATE. In safe sequence: ");
for (int i=0; i<n; i++) {
    printf("P%d", safeSequence[i]);
    if (i!=n-1) printf(" -> ");
}
printf("\n");
return 0;
}

```

Q/P

Enter no of processes: 5

Enter no of resources: 3

Enter allocation matrix (5x3)

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter	Max	
7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

Enter	Availch	
3	3	2

System is
Safe Sequence

UNSAFE
(OK). In

ATE. In

)

);

Enter Max Matrix (5x3)

7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

Enter Available Resources (3)

3	3	2
---	---	---

System is in a SAFE STATE

Safe Sequence: $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$

28/11/26

----- RESOURCE REQUEST PART -----

```
int process;
printf("Enter process number making request (0 to x.d);
scanf("%d", &process);
```

```
int Request[m];
printf("Enter request vector: \n");
for(int j=0; j<m; j++)
    scanf("%d", &Request[j]);
```

```
for(int j=0; j<m; j++)
    if(Request[j] > Need[process][j])
        printf("Error: Process has exceeded its
        maximum claim. \n");
    return 0;
```

```
for(int j=0; j<m; j++)
    if(Request[j] > Available[j])
        printf("Resources not available. Process
        must wait. \n");
    return 0;
```

```
for(int j=0; j<m; j++)
    Available[j] -= Request[j];
    Allocation[process][j] += Request[j];
    Need[process][j] -= Request[j];
```

D/L

Enter number of
Enter number of
Enter allocation

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter Max Mark

7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

Enter Available

3	3	2
---	---	---

Enter process v

Enter request

1	0	2
---	---	---

Request can
Safe Sequence

making request (0 to 4);
n-1);

Diff number of processes : 5
Enter number of resources : 3
Enter allocation Matrix (5x3):

Enter
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix (5x3)

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resource (3):

3 3 2

Enter process number making request (0 to 4): 1

Enter request vector:

1 0 2

Request can be granted. System is SAFE.

Safe Sequence : P1 → P3 → P4 → P0 → P2

Program 7

b) Deadlock Detection

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main () {
```

```
    int n, m;
```

```
    int alloc[MAX][MAX], request[MAX][MAX];
```

```
    int avail[MAX];
```

```
    int finish[MAX];
```

```
    int i, j, count = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resource types: ");
```

```
    scanf("%d", &m);
```

```
    printf("\n Enter Allocation Matrix: \n");
```

```
    for (i=0; i<n; i++) {
```

```
        for (j=0; j<m; j++) {
```

```
            scanf("%d", &alloc[i][j]);
```

```
        }
```

```
    printf("\n Enter Request Matrix: \n");
```

```
    for (i=0; i<n; i++) {
```

```
        for (j=0; j<m; j++) {
```

```
            scanf("%d", &request[i][j]);
```

```
        }
```

```
}
```

```
printf("\n Enter\n");  
for (j=0; j<m; j++) {  
    scanf("%d", &request[i][j]);  
}
```

```
for (i=0; i<n; i++) {  
    int flag = 0;  
    for (j=0; j<m; j++) {  
        if (request[i][j] >= alloc[i][j]) {  
            flag = 1;  
            break;  
        }  
    }  
    if (flag == 0) {  
        finish[i] = 1;  
        count++;  
    }  
}
```

```
while (count < n) {  
    int flag = 0;  
    for (i=0; i<n; i++) {  
        if (finish[i] == 0) {  
            for (j=0; j<m; j++) {  
                if (request[i][j] >= alloc[i][j]) {  
                    flag = 1;  
                    break;  
                }  
            }  
            if (flag == 0) {  
                finish[i] = 1;  
                count++;  
            }  
        }  
    }  
}
```

```
printf("\n Available Resources: \n");  
for (i=0; i<m; i++) {  
    printf("%d ", avail[i]);  
}
```



```

printf("\nEnter Available Resources: \n");
for (j=0; j<m; j++) {
    scanf("%d", &avail[j]);
}

```

```

for (i=0; i<n; i++) {
    int flag = 0;
    for (j=0; j<m; j++) {
        if (alloc[i][j] != 0) {
            flag = 1;
            break;
        }
    }
}

```

```

if (flag == 0)
    finish[i] = 1;
else
    finish[i] = 0;
}

```

```

while (count < n) {
    int found = 0;

```

```

    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int possible = 1;

```

```

            for (j=0; j<m; j++) {
                if (request[i][j] > avail[j]) {
                    possible = 0;
                    break;
                }
            }

```

```

            if (possible) {
                for (j=0; j<m; j++) {
                    avail[j] += alloc[i][j];

```

```

                finish[i] = 1;
            }
}

```

```

        found = 1;
        count++;
    }
    if (found == 0)
        break;
}

int deadlock = 0;
printf("In Processes in Deadlock : ");
for (i = 0; i < n; i++) {
    if (finish[i] == 0) {
        printf("p%d", i);
        deadlock = 1;
    }
}

if (deadlock == 0)
    printf("None (No Deadlock)");

printf("\n");
return 0;

```

O/P

Enter number of processes: 3
 Enter number of resource types: 2

Request

Enter	1
1	1
1	1
1	1
Enter	Available
0	0
Processes in	1

number

Enter	number
Enter	number
Enter	Allocation

1	0
0	1
1	1

Request

Enter	Request
0	0
1	0
0	1

available

Enter	available
1	1

Process in

Request Matrix:

Enter
2
1
1
1

Available Resources:

Enter
0
0

Processes in Deadlock: P0 P1 P2

Enter number of processes: 3

Enter number of resource types: 2

Enter Allocation Matrix:

1 0
0 1
1 1

Enter Request Matrix:

0 0
1 0
0 1

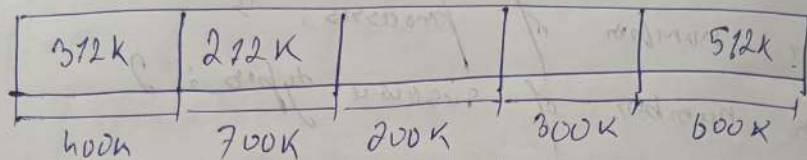
Enter available Resources:

1 1

Process in Deadlock: None (No Deadlock)

~~asked deadline first~~
~~not mentioned scheduling~~
 Give a free memory partition
 400K, 700K, 200K, 300K & 600K
 & how could would each of first fit, best fit and
 worse fit algorithm work to place processes
 of 212K, 512K, 312K, 526K
 P1, P2, P3, P4

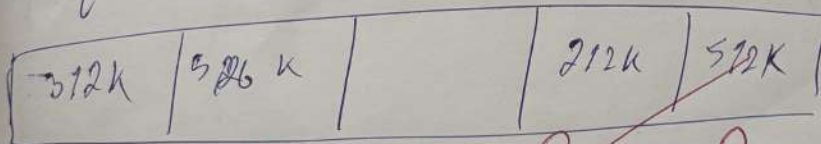
Worst fit



526 K cannot be allocated with memory

~~First fit~~

~~Best fit~~



? ? ? ? ?
 (under the first four cells of the memory diagram)

Incomplete

Program 8 memory
 contiguous
 #include <stdio.h>
 void fit (int b)

{
 int a[mp];
 for (int i = 0;
 for (int i = 0;
 {
 int

for (i
 {

if (i
 {

printf ("")
 for (int

Program 8
Contiguous Memory Allocation Technique

#include <stdio.h>

void fit (int b[], int nb, int p[], int np, int type)

{ int a[np];

for (int i=0; i<np; i++) a[i]=-1;

for (int i=0; i<np; i++)

{ int idx = -1;

for (int j=0; nb; j++)

{ if (b[j] >= p[i])

{ if (type == 0) { idx = j; break; }

if (idx == -1)

{ (type == 1 && b[j] < b[idx]) ||

(type == 2 && b[j] > b[idx])

idx = j;

}

if (idx != -1)

a[i] = idx;

b[idx] -= p[i];

}

printf (" \n PNo | + PSize | + Block \n ");

for (int i=0; i<np; i++)

printf ("%d | + %d | +", i+1, p[i];

if (a[i] != -1)

printf ("%d \n", a[i]+1);

```

else
    printf("NA\n");
}

int main () {
    int nb, np;
    printf("Enter no. of blocks: ");
    scanf("%d", &nb);
    int b1[nb], b2[nb], b3[nb];
    printf("Enter block sizes: \n");
    for (int i=0; i<nb; i++) {
        scanf("%d", &b1[i]);
        b2[i] = b3[i] = b1[i];
    }

    printf("Enter no. of processes: ");
    scanf("%d", &np);

    int p[np];
    printf("Enter process sizes: \n");
    for (int i=0; i<np; i++)
        scanf("%d", &p[i]);

    printf("\n FIRST FIT ");
    fit(h2, nb, p, np, 0);

    printf("\n BEST FIT ");
    fit(h2, nb, p, np, 1);

    printf("\n WORST FIT ");
    fit(h3, nb, p, np, 2);

    return 0;
}

```

Output:

Enter	no. of blocks
Enter	500
100	
Enter	no. of processes
Enter	4
212	

FIRST FIT

PNo	
1	
2	
3	
4	

BEST FIT

PNo	
1	
2	
3	
4	

WORST FIT

PNo	
1	
2	
3	
4	

Output:

Enter no. of blocks: 5
 Enter block sizes:
 100 500 200 300 600

Enter no. of processes: 4
 Enter process sizes:
 212 417 112 426

FIRST FIT

PNo	PSize	Block
1	212	2
2	417	5
3	112	2
4	426	NA

BEST FIT

PNo	PSize	Block
1	212	4
2	417	2
3	112	3
4	426	5

WORST FIT

PNo	PSize	Block
1	212	5
2	417	2
3	112	5
4	426	NA

8K
 125

Program-9

C program to simulate page replacement algorithm
1) FIFO
2) LRU
3) Optimal.

```
#include <stdio.h>
```

```
#define MAX 50
```

```
int search (int p[], int n, int key) {  
    for (int i=0; i<n; i++)  
        if (p[i] == key)  
            return 1;  
    return 0;  
}
```

```
void fifo (int p[], int n, int m) {  
    int f[MAX], i, j, k=0, faults=0;  
    for (i=0; i<m; i++) f[i] = -1;  
    printf ("In FIFO: \n");  
    for (i=0; i<n; i++) {  
        if (!search (f, m, p[i])) {  
            f[k] = p[i];  
            k = (k+1) % m;  
            faults++;  
        }  
        for (j=0; j<m; j++)  
            printf ("%d", f[j]);  
        printf ("\n");  
    }  
    printf ("Page Faults = %d\n", faults);  
}
```

```
void lru (int p[], int n, int m) {  
    int f[MAX], time f[MAX], i, j, pos, min, cnt=0,  
    faults=0;
```



```

for (i=0; i<m; i++)
    p[i] = time[i] = -1;
printf("\n LRU: \n");
for (li=0; i<n; i++) {
    if (search(f, m, p[i]) < K)
        for (j=0; j<m; j++)
            if (f[j] == p[i])
                time[j] = ++cnt;
    else {
        min = time[0], pos=0;
        for (j=2; j<m; j++)
            if (time[j] < min)
                min = time[j], pos=j;
        f[pos] = p[i];
        time[pos] = ++cnt;
        faults++;
    }
    for (j=0; j<m; j++)
        printf("%d", f[j]);
    printf("\n");
}

```

```

printf("Page Faults = %d\n", faults);

```

```

void optimal (int p[], int n, int m) {
    int f[MAX], i, j, K, pos, len, faults=0;
    for (i=0; i<m; i++) f[i] = -1;
    printf("\n optimal : \n");
}

```

```

for (i=0; i<n; i++) {
    if (!search (f, m, p[i])) {
        pos = -1;
        for (j=0; j<m; j++) {
            for (k=i+1; k<n; k++)
                if (f[j] == p[k]) track;
            if (k > pos) {
                pos = k;
                ppos = j;
            }
        }
        if (pos == -1) pos = 0;
        f[ppos] = p[i];
        faults++;
    }
    for (j=0; j<m; j++)
        printf ("%d", f[j]);
    printf ("\n");
}

```

```

printf ("Page Faults = %d\n", faults);

int main () {
    int p[MAX], n, m, i;
    printf ("Enter number of pages: ");
    scanf ("%d", &n);
    printf ("Enter pages: ");
    for (i=0; i<n; i++)
        scanf ("%d", &p[i]);
    printf ("Enter frame size: ");
}

```

scanf ("%d", &n);
 printf ("Enter number of pages: ");
 scanf ("%d", &n);
 printf ("Enter pages: ");
 optional (p)
 return 0;

O/P

Enter number of
 pages: 1
 Enter frame size

FIFO :		
1	1	-1
1	2	-1
1	2	3
4	2	3
4	2	3
4	1	2
5	1	2
5	1	2
5	1	2
5	3	2
5	3	4
5	3	4

Page Faults =


```

scanf ("%d", &n);
fifo (p, n, m)
{
    int i, j, k;
    optimal (p, n, m);
    return 0;
}

```

O/P

Enter number of pages: 12
 Enter pages: 1 2 3 4 1 2 5 1 2 3 4 5
 Enter frame size: 3

FIFO :

1	1	-1
1	2	-1
1	2	3
4	2	3
4	2	3
4	1	2
5	1	2
5	1	2
5	1	2
5	3	2
5	3	4
5	3	4

Page Faults = 9

LRU:

1	-1	-1
1	2	-1
1	2	3
4	2	3
4	2	3
4	2	2
5	1	2
5	2	2
5	2	2
3	1	2
3	4	2
3	4	5

Page Faults = 10

Optimal:

1	-1	-1
1	2	-1
1	2	3
1	2	4
2	2	4
1	2	4
1	2	5
1	2	5
1	2	5
3	2	5
4	2	5
4	2	5

Page Faults = 7

26/12/26